LLVM DEV MEETING 2026 · MOJO WORKSHOP

How we write Mojo in Mojo

How the Mojo standard library provides a nice interface over MLIR constructs

Nathan Ward · Joe Loser

Mojo Libraries · Modular

Modular

OVERVIEW

Where we're going

1 **Raw MLIR in Mojo**
Our program with no stdlib, and the built-ins it uses

2 **Wrapping MLIR**
Build our own Int, then compare it with the real stdlib

3 **Leaves and aggregates**
The few types that touch MLIR, and everything built on them

4 **Changing what's underneath**
Swapping Bool's representation, and where the boundary leaks

5 **Types that compute**
Constant folding inside the type system

6 **What it costs**
Compile time, strings, debugging, and escape hatches

01 · RAW MLIR IN MOJO

Building Int from scratch

We'll write our own Int in Mojo, without the standard library

The goal: ordinary Mojo

```
def sum_of_squares() -> Int:
    var a: Int = 42
    var b: Int = 8

    var asqrd = a * a
    var bsqrd = b * b
    return asqrd + bsqrd
```

What we want to write.

Step 0 · Write it directly in MLIR

```
def sum_of_squares() -> __mlir_type.index:
    var a: __mlir_type.index = __mlir_attr.`42 : index`
    var b: __mlir_type.index = __mlir_attr.`8 : index`

    var asqrd = __mlir_op.`index.mul`(a, a)
    var bsqrd = __mlir_op.`index.mul`(b, b)
    return __mlir_op.`index.add`(asqrd, bsqrd)
```

This compiles and runs today.

index is MLIR's built-in integer type, as wide as the target's machine word, like size_t.

The ops come from MLIR's upstream index dialect.

MLIR index dialect: mlir.llvm.org/docs/Dialects/IndexOps

Built-ins for writing MLIR

`__mlir_type`	Name an MLIR type	`__mlir_type.index`
`__mlir_attr`	A compile-time constant	`__mlir_attr.\`42 : index\``
`__mlir_op`	Invoke an MLIR op by name	`__mlir_op.\`index.add\`(a, b)`

Public, documented syntax. This is how the stdlib reaches MLIR.

02 · WRAPPING MLIR

Users should write Mojo, not MLIR

02 · WRAPPING MLIR

Step 1 · Let's make our own Int

```
struct Int:
    var _mlir: __mlir_type.index

    def __init__(out self, _mlir: __mlir_type.index):
        self._mlir = _mlir
```

The MLIR type is spelled in one place. Code holding an Int never says index again.

02 · WRAPPING MLIR

Our Int can't be built from 42 yet

```
$ mojo build main.mojo
error: no matching function in initialization
    var a = Int(42)
            ~~~^~~~
```

WHAT IS 42? · AN INTLITERAL

```
struct IntLiteral[value: __mlir_type.`!pop.int_literal`]
```

A literal has its own type. Its value lives in that type, as a compile-time parameter.

So give Int an __init__ that takes an IntLiteral.

02 · WRAPPING MLIR

Our Int can't multiply yet

```
$ mojo build main.mojo
error: 'Int' does not implement the '__mul__' method
    var asqrd = a * a
                ~ ^
```

THE OP WE NEED · MLIR INDEX DIALECT

`index.mul (index::MulOp)`

Takes two index values and computes their product.

```
%c = index.mul %a, %b
(index, index) -> index
```

So give Int a __mul__ that calls index.mul.

index.mul docs: mlir.llvm.org/docs/Dialects/IndexOps

Step 2 · Let 42 become an Int

```
struct Int:
    @implicit
    def __init__(out self, literal: IntLiteral[_]):
        self._mlir = __mlir_attr[
            `#kgen.cast_to_builtin<#pop.int_literal_convert<`,
            literal.value,
            `> : !kgen.scalar<index>> : index`,
        ]
```

`#pop.int_literal_convert`
turns the literal into a !kgen.scalar<index>

`#kgen.cast_to_builtin`
casts that to MLIR's builtin index

Step 3 · Hide the ops behind methods

```
struct Int:
    var _mlir: __mlir_type.index

    def __add__(self, rhs: Int) -> Int:
        var mlir = __mlir_op.`index.add`(self._mlir, rhs._mlir)
        return Int(_mlir=mlir)

    def __mul__(self, rhs: Int) -> Int:
        var mlir = __mlir_op.`index.mul`(self._mlir, rhs._mlir)
        return Int(_mlir=mlir)
```

The goal program, revisited

```
def sum_of_squares() -> Int:
    var a: Int = 42
    var b: Int = 8
    var asqrd = a * a
    var bsqrd = b * b
    return asqrd + bsqrd
```

Try it: godbolt.org/z/ec8xhvjsn

From Mojo to MLIR

1 · You write

```
var a: Int = 42
var b: Int = 8
var asqrd = a * a
var bsqrd = b * b
return asqrd + bsqrd
```

2 · Desugared to method calls

```
var a = Int.__init__(42)
var b = Int.__init__(8)
var asqrd = a.__mul__(a)
var bsqrd = b.__mul__(b)
return asqrd.__add__(bsqrd)
```

3 · After inlining: the program we started with

```
var a: __mlir_type.index = __mlir_attr.`42 : index`
var b: __mlir_type.index = __mlir_attr.`8 : index`
var asqrd = __mlir_op.`index.mul`(a, a)
var bsqrd = __mlir_op.`index.mul`(b, b)
return __mlir_op.`index.add`(asqrd, bsqrd)
```

02 · WRAPPING MLIR

The real standard library

Bool and Int are built the same way as the Int we just wrote

The real Int is SIMD, which computes its MLIR type

```
comptime Int = Scalar[DType.int]
comptime Scalar = SIMD[_, 1]

struct SIMD[dtype: DType, length: SIMDLength]:
    comptime _mlir_type = __mlir_type[
        `!kgen.simd<`,
        Self.length._mlir_value,
        `, `,
        Self.dtype._mlir_value,
        `>`,
    ]
    var _mlir_value: Self._mlir_type
```

The MLIR type is built from the parameters.

SIMD[DType.float32, 4] stores a !kgen.simd<4, f32>.

One pop.add is + for every Int, Float64, and vector.

Zero run-time cost: the wrapper inlines away

Mojo

```
var a: Int = 42
var b: Int = 8
var c = a + b
```

Parsed: a call into the library

```
%c = call @Int.__add__(%a, %b)
```



After inlining: no call, no struct

```
%c = pop.add %a, %b : !kgen.scalar<index>
```



LLVM IR

```
%c = add i64 %a, %b
```

Three layers of library code become one instruction. Zero run-time cost.

A handful of leaf types wrap MLIR

LEAF	WRAPS	HIDES	YOU WRITE
Bool	`!kgen.scalar<bool>`	`pop.simd.xor` · `pop.simd.and`	`not` · `&` · `|`
SIMD so Int, Float64, …	`!kgen.simd<N, dtype>`	`pop.add` · `pop.simd.splat`	`+` · `*` · `==`
Pointer	`!kgen.pointer<T, space>`	`pop.load` · `pop.store`	`p[i]` · `p.load()`
VariadicPack	`!lit.ref.pack<…>`	`lit.ref.pack.extract`	`*args: *Ts`
TypeList	`!kgen.param_list<…>`	`#kgen.param_list.get`	`tl[i]` · `len(tl)`

Most of the stdlib's MLIR lives here.

Everything else is built from leaves

```
struct List[T]:
    var data: Pointer[Self.T, _]
    var length: Int
    var capacity: Int
```

`Pointer` + `Int` + `Int`

List is three leaves and no MLIR.

Most MLIR is concentrated in the leaf types.

Where the MLIR lives

Your code Operators, methods, literals	none needed
Aggregates List · Dict · String · Optional	a little, or none
Leaves Bool · SIMD · Pointer · DType · Atomic	most of it

Users don’t have to see MLIR.

It’s concentrated in a few core types, and every one of them hides it behind ordinary methods.

04

Changing what's underneath

Developers only use the wrapper, so we can swap the MLIR behind it

04 · CHANGING WHAT'S UNDERNEATH

In May, Bool moved off i1

Before

`Bool → i1`

`SIMD[DType.bool, N] → its own lanes`

Two representations of one boolean



After

`Bool → !kgen.scalar<bool>`

`SIMD[DType.bool, N] lanes → same`

One representation, shared

Why: the compile-time folder already produced `!kgen.scalar<bool>` for `and`, `where` clauses and `conforms_to`, so every folded Bool needed a cast back to `i1`.

User code didn't change.

04 · CHANGING WHAT'S UNDERNEATH

But a library change touched the compiler

23

compiler source files changed in the "library" PR, not counting tests

- `Elaborator`
- `CheckLifetimes`
- `MojoLLDB` · the debugger
- `KernelDeclAdaptor` · the graph compiler

+ a new compiler hook

Why the compiler changed: it knows Bool's field by name



Other stdlib names the compiler depends on:

Fields & hooks	`_mlir_value` `__mlir_bool__` `__mlir_index__` `__mlir_i1__`
Literal hooks	`__list_literal__` `__dict_literal__` `__set_literal__` `__slice_literal__`
Types & traits	`Bool` · `Int` · `IntLiteral` · `String` · `Tuple` · `Error` · `AnyType` · …

The abstraction holds for users. These are Mojo's equivalent of Rust's lang items.

Mojo/lib/MojoParser/IREmitter.cpp · names collected from Mojo/lib

Encapsulation: a public API over private MLIR

Public API

What users write, and we keep stable

`Bool · Int · SIMD · List`

`x + y · not x · p[i]`

Implementation details

The MLIR underneath, which we can change

`pop · kgen · lit dialects`

`!kgen.scalar<bool> · pop.simd.xor`

"internal implementation details of the compiler and may change without notice" (Mojo's inline-MLIR docs)

That boundary is why Bool could move from i1 to !kgen.scalar<bool> without breaking anyone.

Mojo/docs/site/reference/inline-mlir.mdx

05

Types that compute

Library code doing constant folding inside the type system

A value can live in a type

```
struct T[n: Int]:
    def __init__(out self): pass

def takes_one(x: T[1]): pass

takes_one(T[1]())   # ok
takes_one(T[99]())  # error: T[99] is not T[1]
```

`T[1]` ≠ `T[99]`

n is a compile-time parameter, not a field. It isn't stored in the value at all.

The type checker compares the values to decide if two types match.

So the type checker has to evaluate Int values. What about T[a + b]?

A signature can compute its return type

Illustrative

```
def concat[a: Int, b: Int](
    x: T[a], y: T[b]
) -> T[a + b]: ...

var r = concat(T[12](), T[8]())
# r: T[20]
```



The type checker inlines Int.__add__ into the parameter expression, down to the MLIR add, then constant-folds it at each call site.

The type checker folds; errors keep your spelling

You write `T[a + b]`

↓

Compiler builds `T[Int{pop.add(a._mlir_value, b._mlir_value)}]`

↓

Folds to `T[20]` when a = 12, b = 8

Errors say

```
error: … cannot be converted from 'T[(a + b)]' to 'T[Int(1)]'
```

Where this folding happens in the pipeline



Earlier approach

Ran the interpreter on unchecked parser IR

Today

Expressions like a + b fold here, on checked types

9:30 talk

add[p] folded during elaboration

Earlier approach: Mojo/docs/stdlib/internal/mlir.md

06 · WHAT IT COSTS

The costs of zero run-time overhead

↩ the raw MLIR program

Strings, not types

MLIR is spelled in backtick strings the Mojo compiler can't check. Typos surface late; no autocomplete.

```
`!kgen.scalar<bool>`
```

↩ the lowering slides

Compile time

Every + on an Int is a call the compiler must inline away, on types nearly every line of Mojo uses.

```
a.__add__(b) → pop.add
```

↩ zero run-time cost

Debugger noise

Step into x + y and you'd land in wrapper methods. We hide them by hand, about 440 times.

```
@inline(.nodebug)
```

When there's no MLIR op for it

llvm_intrinsic

No MLIR op, but LLVM has an intrinsic: call it by name.

```
def pop_count(x: SIMD) -> SIMD:
    return llvm_intrinsic[
        "llvm.ctpop", type_of(x)
    ](x)
```

Count the set bits → one popcnt instruction

inlined_assembly

No intrinsic either: write the target instruction itself.

```
var packed = inlined_assembly[
    "cvt.rp.ue8m0x2.f32 $0, $1, $2;",
    UInt16, constraints="=h,f,f",
](hi, lo)
```

float32 → float8 (e8m0) on NVIDIA GPUs, as one PTX instruction

RECAP

What we covered

01 Mojo can write raw MLIR directly with __mlir_type, _attr and _op

02 The stdlib wraps it: one field per type, one op per method, and it inlines back to a single add i64

03 A few leaf types hold most of the MLIR; everything else is structs of leaves

04 Encapsulation lets us change the MLIR underneath, except at names the compiler knows

05 Types can compute: T[a + b] folds to T[20], but errors keep what you wrote

06 Free at run time, paid for in compile time, string syntax and debugger work

Questions?

Nathan Ward · Joe Loser

`github.com/modular/modular · Mojo/`

The Mojo Libraries team is hiring

Accelerator abstractions · core standard library · low-level hardware drivers

`modular.com/careers`

Modular